ES6 Fundamentals, Typescript

1. Scoping using Let and Const

- `let` and `const` provide block-level scoping (unlike `var` which is function-scoped).
- `const` cannot be reassigned after declaration.

```
// var (function-scoped)
if (true) {
  var x = 10;
}
console.log(x); // 10 (leaks outside block)

// let (block-scoped)
if (true) {
  let y = 20;
  console.log(y); // 20
}
console.log(y); // ReferenceError: y is not defined

// const (block-scoped, immutable reference)
const PI = 3.14;
PI = 3.14159; // TypeError: Assignment to constant variable.
```

2. Spread Syntax and Rest Parameters

- **Spread (`...`)** expands an array/object into individual elements.
- **Rest (`...`)** collects remaining arguments into an array.

```
// Spread (arrays)
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

// Spread (objects)
const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }

// Rest parameters
function sum(...numbers) {
  return numbers.reduce((acc, num) => acc + num, 0);
```

```
}
sum(1, 2, 3); // 6
```

3. Default Parameters

- Allows setting default values for function parameters.

```
function greet(name = "Guest") {
  console.log(`Hello, ${name}!`);
}
greet(); // Hello, Guest!
greet("Alice"); // Hello, Alice!
```

4. Array/Object Destructuring

- Unpacks values from arrays/objects into variables.

```
// Array destructuring
const [first, second] = [10, 20];
console.log(first); // 10

// Object destructuring
const { name, age } = { name: "Bob", age: 30 };
console.log(name); // Bob
```

5. Arrow Functions

- Shorter syntax and lexical `this` binding.

```
const add = (a, b) => a + b;
console.log(add(2, 3)); // 5

// Lexical `this`
const person = {
  name: "Alice",
  greet: function() {
    setTimeout(() => console.log(`Hi, ${this.name}!`), 1000);
  }
};
person.greet(); // Hi, Alice! (uses surrounding `this`)
```

6. Template Literals

- Multi-line strings with embedded expressions.

```
const name = "John";
const age = 25;
console.log(`Name: ${name}, Age: ${age}`); // Name: John, Age: 25
```

7. Promises

- Handles asynchronous operations.

```
const fetchData = () => {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("Data fetched!"), 2000);
  });
};

fetchData()
  .then(data => console.log(data)) // "Data fetched!"
  .catch(err => console.error(err));
```

---

8. Async / Await

- Syntactic sugar over Promises.

```
async function getData() {
  try {
    const data = await fetchData();
    console.log(data); // "Data fetched!"
  } catch (err) {
    console.error(err);
  }
}
getData();
```

9. Classes and Objects

- Syntactic sugar over prototype-based inheritance.

javascript

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}!`);
  }
}

const alice = new Person("Alice");
alice.greet(); // Hello, Alice!
```

Node.js and NPM Basics

1. Node.js Introduction

- JavaScript runtime built on Chrome's V8 engine.
- Enables server-side JS execution.

2. Package Managers (npm, yarn)

- Manages dependencies in a project.

3. npm init & npm install

- `npm init` creates a `package.json`.
- `npm install <package>` installs dependencies.

```
npm init -y  # Creates package.json
npm install express  # Installs Express.js
```

TypeScript

1. Installing TypeScript

```
npm install -g typescript
```

2. Compiling TypeScript

bash

```
tsc app.ts  # Generates app.js
```

3. Setup TypeScript Project

```
tsc --init  # Creates tsconfig.json
```

4. Basic Data Types

```
let name: string = "Alice";
let age: number = 30;
let isActive: boolean = true;
let scores: number[] = [90, 85, 95];
```

5. Classes

```
class User {
  constructor(public name: string, private age: number) {}
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}
```

6. Interfaces

```
interface Person {
  name: string;
  age: number;
}

const alice: Person = { name: "Alice", age: 25 };
```

7. Inheritance

```
class Employee extends User {
  constructor(name: string, age: number, public role: string) {
    super(name, age);
  }
}
```

- **ES6+** introduces modern JS features like `let/const`, arrow functions, promises, etc.
- **Node.js** allows JS to run outside the browser.
- **TypeScript** adds static typing to JavaScript for better tooling and safety.